# Real-Time Linux OS Using PREEMPT_RT Kernel with

# 5G Log Transfer

*Report submitted to the SASTRA Deemed to be University in partial fulfillment of the requirements for the award of the degree of*

**B. Tech. Electronics & Communication Engineering**

**(Cyber Physical Systems)**

*Submitted by*

**Archith Kumar J**

**(Reg. No.: 127160005)**
**Josith G**
**(Reg. No.: 127160023)**

**May 2026**




## SCHOOL OF ELECTRICAL & ELECTRONICS ENGINEERING

THANJAVUR, TAMIL NADU, INDIA - 613 401

SASTRA

ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act, 1956)

THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

**SCHOOL OF ELECTRICAL & ELECTRONICS ENGINEERING**
**THANJAVUR - 613 401**

**Bonafide Certificate**

This is to certify that the report titled **"Real-Time Linux OS Using PREEMPT_RT Kernel with 5G Log Transfer"** submitted in partial fulfillment of the requirements for the award of the degree of B. Tech. ELECTRONICS & COMMUNICATION ENGINEERING (CYBER PHYSICAL SYSTEMS) to the SASTRA Deemed to be University, is a bonafide record of the work done by **Archith Kumar J (127160005)** and **Josith G (127160023)** during the Sixth semester of the academic year 2025-26, in the **School of Electrical & Electronics Engineering**, under my supervision. This report has not formed the basis for the award of any degree, diploma, associateship, fellowship or other similar title to any candidate of any University.

**Signature of Project Supervisor** : 6/5/26

**Name with Affiliation** : Dr. Parvathy A, Senior Assistant Professor, SEEE.

**Date** : 06.05.2026

Project *Viva voce* held on 07.05.2026

**Examiner 1** **Examiner 2**

SASTRA

ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act, 1956)

40 YEARS

THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

**SCHOOL OF ELECTRICAL & ELECTRONICS ENGINEERING**
**THANJAVUR - 613 401**

## Declaration

We declare that the thesis titled **"Real-Time Linux OS Using PREEMPT_RT Kernel with 5G Log Transfer "** submitted by us is an original work done by us under the guidance of Dr. Parvathy A, Senior Assistant Professor, School of Electrical and Electronics Engineering, SASTRA Deemed to be University during the Sixth semester of the academic year 2025-26, in the School of Electrical and Electronics Engineering. The work is original and wherever we have used materials from other sources, we have given due credit and cited them in the text of the thesis. This thesis has not formed the basis for the award of any degree, diploma, associate-ship, fellowship or other similar title to any candidate of any University.

Signature of the candidate(s) : J. Archith Kumar Josithgaresan

Name of the candidate(s) :Archith Kumar J (127160005)

Josith G (127160023)

Date : 06.05.2026

## Acknowledgements

We would like to express our sincere gratitude to our Honourable Chancellor, **Prof. R. Sethuraman**, for providing us with the opportunity and the necessary infrastructure to carry out this project as part of our curriculum.

We extend our heartfelt thanks to our honorable Vice-Chancellor, **Dr. S. Vaidhyasubramaniam**, and **Dr. S. Swaminathan**, Dean, Planning & Development, for their constant encouragement and strategic support throughout our academic journey. We would also like to thank **Dr. R. Chandramouli**, Registrar for providing us with the opportunity to undertake this project.

We express our sincere appreciation to **Dr. K. Thenmozhi**, Dean, SEEE; **Dr. R. John Bosco Balaguru**, Dean (Sponsored Research); and **Dr. A. Krishnamoorthy**, Associate Dean (Academics), for introducing and facilitating this project as part of our curriculum.

We would like to extend our sincere thanks to **Dr. James A. Baskaradas**, Professor, for his valuable support and encouragement throughout the course of this project.

We express our profound gratitude to our **Dr. Parvathy A**, Senior Assistant Professor guide from School of Electrical & Electronics Engineering, whose unwavering support and guidance have been pivotal to the success of our project.

We sincerely thank the Project Review Panel Members **Dr. Hemavathi N** and **Dr. Valarmathi R** Senior Assistant Professors from School of Electrical & Electronics Engineering, for their valuable suggestions and constructive feedback throughout the course of this project. Their insightful observations greatly contributed to improving the quality and depth of our work. We extend our appreciation to all the teaching and non-teaching staff of the School of Electrical & Electronics for their support and assistance, directly or indirectly, in the completion of this project.

Finally, we gratefully acknowledge the encouragement and support of our family and friends, which played a vital role in the successful completion of this project.

## Abstract

This project creates a real-time edge computing system in which a Raspberry Pi 4 sends live system log messages to a MEC server over a Private 5G Standalone (SA) network. Standard Linux operating systems have scheduling jitter, which means that high-priority tasks can be delayed in ways that are hard to predict. This makes them not good for applications that need to be done quickly. To fix this, a real-time Linux kernel with the PREEMPT_RT patch was manually compiled and installed. This made it possible to schedule tasks in a predictable way and let high-priority processes run with very little delay.

Initially, Buildroot was used to create a lightweight operating system, producing a compact 45 MB image with low memory usage and a boot time of around 6 seconds. However, the system was later migrated to Raspberry Pi OS because Buildroot lacks package manager, making it difficult to install and manage the required software for the 5G hardware module. So, the PREEMPT_RT kernel was manually patched and compiled on Raspberry Pi OS to enable real-time performance.

Flask and Socket were used to make a web-based monitoring system on the receiver side. IO gets log messages over the network and shows them in real time on a browser that can be accessed from any device. Using cyclictest, we found that the worst-case latency went down from about 149 microseconds to 45 microseconds, which is more than three times better. Also, the end-to-end delay from making a log on the Raspberry Pi to seeing it in the browser was measured to be between 16 and 28 milliseconds. This shows that the system meets the requirements for real-time monitoring applications in smart infrastructure, healthcare, and industrial applications.


**Specific Contribution**

- Implemented the sender-side Python script on the Raspberry Pi that captures live system logs using journalctl and transmits each log entry as a UDP data packet over the 5G network to the server
- Led the Buildroot operating system configuration process including kernel setup using menuconfig, package selection for required drivers and tools, and bootable SD card image generation and deployment

**Specific Learning**

- Gained practical experience in embedded Linux operating system development using Buildroot
- Learned to build multi-threaded Python network applications combining UDP packet reception and real-time WebSocket broadcasting to multiple browser clients simultaneously
- Understood basics of private 5G SA Network setup.

**Technical Limitations and Ethical Challenges Faced**

- The Buildroot package ecosystem did not include the software needed to configure the SIM8200EA-M2 5G module, requiring a switch to manual kernel compilation on the full operating system
- Higher memory and storage overhead when running the PREEMPT_RT kernel on the full Raspberry Pi OS compared to the minimal Buildroot image
- Privacy considerations arise when system log data is transmitted over a wireless network, as logs may contain sensitive process and user information

***Keywords:*** *PREEMPT_RT, Real-Time Linux, 5G Standalone, Buildroot, Edge Computing, Flask, Socket.IO, Raspberry Pi, UDP, Latency*

J. Archith Kumar

**Signature of the Student**

**Name:** Archith Kumar J

**Signature of Guide**

**Name:** Dr. Parvathy A

# Abstract

This project creates a real-time edge computing system in which a Raspberry Pi 4 sends live system log messages to a MEC server over a Private 5G Standalone (SA) network. Standard Linux operating systems have scheduling jitter, which means that high-priority tasks can be delayed in ways that are hard to predict. This makes them not good for applications that need to be done quickly. To fix this, a real-time Linux kernel with the PREEMPT_RT patch was manually compiled and installed. This made it possible to schedule tasks in a predictable way and let high-priority processes run with very little delay.

Initially, Buildroot was used to create a lightweight operating system, producing a compact 45 MB image with low memory usage and a boot time of around 6 seconds. However, the system was later migrated to Raspberry Pi OS because Buildroot lacks package manager, making it difficult to install and manage the required software for the 5G hardware module. So, the PREEMPT_RT kernel was manually patched and compiled on Raspberry Pi OS to enable real-time performance.

Flask and Socket were used to make a web-based monitoring system on the receiver side. IO gets log messages over the network and shows them in real time on a browser that can be accessed from any device. Using cyclictest, we found that the worst-case latency went down from about 149 microseconds to 45 microseconds, which is more than three times better. Also, the end-to-end delay from making a log on the Raspberry Pi to seeing it in the browser was measured to be between 16 and 28 milliseconds. This shows that the system meets the requirements for real-time monitoring applications in smart infrastructure, healthcare, and industrial applications.


**Specific Contribution**

- Developed the Multi-access Edge Computing server receiver application using Flask and Socket.IO, including the background UDP listener thread and real-time WebSocket log forwarding to the browser interface
- Performed complete hardware setup including mounting the SIM8200EA-M2 5G module on the Raspberry Pi, configuring network registration, and verifying 5G connectivity before operating system development began

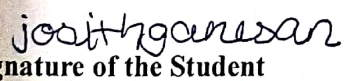
**Specific Learning**

- Gained hands-on experience in cross-compiling the Linux kernel for an ARM64 embedded platform on a host machine and deploying the compiled kernel to the Raspberry Pi through SD card installation
- Learned to measure and interpret real-time operating system performance using cyclictest and to evaluate end-to-end network latency over a live 5G connection between the Raspberry Pi and the server
- Understood basics of private 5G SA Network setup.

**Technical Limitations and Ethical Challenges Faced**

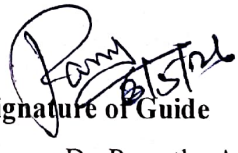
- The Buildroot package ecosystem did not include the software needed to configure the SIM8200EA-M2 5G module, requiring a switch to manual kernel compilation on the full operating system
- Higher memory and storage overhead when running the PREEMPT_RT kernel on the full Raspberry Pi OS compared to the minimal Buildroot image
- Privacy considerations arise when system log data is transmitted over a wireless network, as logs may contain sensitive process and user information

***Keywords:*** *PREEMPT_RT, Real-Time Linux, 5G Standalone, Buildroot, Edge Computing, Flask, Socket.IO, Raspberry Pi, UDP, Latency*



**Signature of the Student**

**Name:** Josith G



**Signature of Guide**

**Name:** Dr. Parvathy A

# List of Figures


| Figure No. | Title | Page No. |
|---|---|---|
| Fig. 4(a) | Overall System Architecture | 7 |
| Fig. 4(b) | Overall System Setup | 7 |
| Fig. 4(c) | Raspberry Pi 4 with SIM8200EA-M2 5G HAT Hardware Setup | 8 |
| Fig. 5(a) | Network Connectivity Test on Raspberry Pi OS | 10 |
| Fig. 5(b) | Buildroot Main menuconfig Menu | 11 |
| Fig. 6(a) | System Information | 16 |
| Fig. 6(b) | SIM in 5G HAT module was not activated | 16 |
| Fig. 6(c) | Raspberry Pi Connected to Network via SIM8200EA 5G HAT Module | 17 |
| Fig. 6(d) | SIM in 5G HAT module was activated | 17 |
| Fig. 6(e) | MEC Server Initialized to Receive Logs | 17 |
| Fig. 6(f) | Logs Sent from Raspberry Pi to MEC Server | 18 |
| Fig. 6(g) | Real-Time Logs Received on Mobile Device | 18 |
| Fig. 6(h-k) | Ping test results | 19 |

## List of Tables


| Table No. | Title | Page No. |
|---|---|---|
| 6 (i) | Comparison between Raspberry Pi OS and Custom Buildroot OS | 20 |
| 6 (ii) | Comparison between Raspberry Pi OS and Custom Real Time OS | 21 |

# Abbreviations

5G – Fifth Generation

CPU – Central Processing Unit

eMBB – Enhanced Mobile Broadband

gNB – Next-Generation Node B

HTTP – Hypertext Transfer Protocol

MEC – Multi-access Edge Computing

NR – New Radio

OS – Operating System

PREEMPT_RT – Real-Time Preemption Patch

SA – Standalone

SIM – Subscriber Identity Module

## Table of Contents


Bonafide Certificate ii
Declaration iii
Acknowledgements iv
Abstract v
List of Figures ix
List of Tables x
Abbreviations xi

**CHAPTER 1 INTRODUCTION**

1.1 Background 1
1.2 Real-Time Systems and need for Determinism 1
1.3 Overview of 5G Standalone (SA) Network 2
1.4 Preempt_RT Real-Time Linux Kernel 2
1.5 Multi-Access Edge Computing (Mec) 3
1.6 Motivation 3
1.7 Problem Statement 3

**CHAPTER 2 LITERATURE REVIEW**

2.1 Performance Assessment of Linux Kernels with PREEMPT_RT on ARM-Based Embedded Devices 4
2.2 Buildroot Manual – Embedded Linux System Build Tool 4
2.3 Linux Kernel Documentation for Raspberry Pi 4
2.4 Performance Evaluation of 5G Standalone Networks for Ultra-Low Latency Applications 5
2.5 A Survey on Mobile Edge Computing: The Communication Perspective 5
2.6 Research Gap Identification 5

**CHAPTER 3 OBJECTIVES**

3.1 Design of Real-Time Linux System 6
3.2 Integration With 5G Network 6
3.3 Implementation of MEC-Based Log Transfer 6
3.4 Performance Evaluation 6

**CHAPTER 4 SYSTEM ARCHITECTURE**

4.1 Overall System Design 7
4.2 Hardware Components 8
4.3 Software Architecture 8
4.4 Data Flow and Communication Model 9

**CHAPTER 5 METHODOLOGY**


| | | |
|---|---|---|
| 5.1 | Hardware Setup and Initial Testing | 10 |
| 5.2 | Buildroot-Based OS Development | 11 |
| 5.3 | Kernel Configuration with Preempt_RT | 12 |
| 5.4 | Cross Compilation and Deployment | 12 |
| 5.5 | Real-Time Log Transmission Mechanism | 14 |


**CHAPTER 6 RESULTS AND DISCUSSION**


| | |
|---|---|
| 6.1 System Verification Results | 16 |
| 6.2 Results of Real-Time Log Transfer Over 5G Network | 16 |
| 6.3 Latency Analysis | 19 |
| 6.4 OS Comparison Table | 20 |


**CHAPTER 7 APPLICATIONS**


| | |
|---|---|
| 7.1 Autonomous Vehicles | 22 |
| 7.2 Industrial Automation | 22 |
| 7.3 Healthcare Systems | 22 |

**CHAPTER 8 CONCLUSION** 24

**REFERENCES** 25

**APPENDIX** 26

# CHAPTER 1

# INTRODUCTION

## 1.1 Background

The evolution of 5G and edge computing have evolved rapidly in today's world , the way that time-critical data is collected, processed, and acted upon in modern systems has changed completely. For example, industrial robots, autonomous vehicles, and remote healthcare devices all have one thing in common; they all require the delivery of data with predictable, bounded delay.

Linux has quickly become the platform of choice for embedded and edge computing due to its open-source nature, support for a broad range of hardware platforms, and the maturity of its networking stack. The standard Linux kernel was designed for throughputs and fairness, not for deterministic timing; it contains protected areas in which preemption is not allowed, resulting in unpredictable latency spikes, termed scheduling jitter, which can reach several milliseconds simply unacceptable for real-time applications that require safety-critical guarantees.

5G Standalone (SA) networks provide higher throughputs, added capability of network slicing, and native support for Multi-access Edge Computing (MEC), resulting in the ability to achieve true end-to-end low latency. In this project, we will be looking at the practical intersection of the performance of PREEMPT_RT real-time Linux and 5G Standalone networks, specifically determining whether a Raspberry Pi 4 can transmit logs from a system to an edge computing server with a measurable and stable degree of bounded latency.

## 1.2 Real-Time Systems and need for Determinism

Real-time systems aren't defined by speed, but rather by predictability. There are two types of real-time systems: hard real-time and soft real-time.

The standard Linux Kernel is not fully preemptible, which means that there are some parts of the kernel where preemption cannot be switched back on. Because of this, high-priority tasks may have to wait when preemption is turned off and delay spikes will occur. Therefore, standard Linux cannot be used for hard real time workloads. The primary measure of how a

real-time OS works from an evaluation standpoint would be the worst case latency, which is measured using a utility known as cyclictest.

### 1.3 Overview of 5G Standalone (SA) Network

5G Standalone networks use a fully native 5G Core that uses standardized network functions, including Access and Mobility Management Function, Session Management Function, User Plane Function, and Policy Control Function. The standardized network functions communicate with each other using a service-based architecture over HTTP/2. Lab setup uses the n78 band (~3.5 GHz) with 100 MHz bandwidth and 30 kHz subcarrier spacing. Using the fully native 5G Core allows for real network slicing, native MEC integration, as well as true end-to-end low latency for Ultra-Reliable Low-Latency Communication (URLLC) slices. Since URLLC slice is unavailable, the Raspberry Pi is connected to the network via a SIM8200EA-M2 5G HAT module, register on the 5G network, get an IP address and communicate with the Multi-access Edge Computing server over the Enhanced Mobile Broadband slice (eMBB).

### 1.4 Preempt_RT Real-Rime Linux Kernel

The PREEMPT_RT patch allows the regular Linux Kernel to be turned into a true preemptive Real-Time OS by making 3 major changes:

- Spinlocks convert into sleeping mutexes which allow for higher priority tasks to preempt other tasks.
- Interrupt handlers convert into kernel threads (threaded IRQs) which can then be scheduled like any other task.
- High-resolution timer support will be enforced so that time can be measured in microseconds.

These changes result in removing virtually every section of the kernel that is intended to block preemption. Using normal Raspberry Pi OS Kernel, cyclictest would report that there was the possibility of seeing worst-case latencies of more than 149 μs. With the addition of PREEMPT_RT, typical numbers will drop below 50 μs. This provides the type of deterministic timing that is necessary for Real-Time Applications.

### 1.5 Multi-Access Edge Computing (MEC)

Multi-access Edge Computing, as defined by European Telecommunications Standards Institute (ETSI), is a method for deploying compute resources close to the device. This provides for lower round trip times between the device and the server performing the processing, and moves the bulk of compute out of the core network. During this project, the Multi-access Edge Computing server is going to run a Python Flask application with Socket.IO and implement a UDP listener that listens for log data sent from a Raspberry Pi over the 5G cellular network, and deliver it in real-time to a web browser.

### 1.6 Motivation

- Increasing need for industrial systems to have real-time telemetry available, while standard Linux cannot ensure this.
- Even though 5G has the potential to deliver low latencies, the delay from the application layer to the physical layer may be impacted much more by the OS scheduling process (jitter), than the network path itself.
- No published literature on how to combine PREEMPT_RT with a 5G SA with native MEC network using a Raspberry Pi
- This project will create a complete practical (end-to-end) reference implementation of real-time edge telemetry.

### 1.7 Problem Statement

**Low-latency** and **deterministic communications** are both requirements of modern industrial and embedded products to perform successfully. 5G Standalone networks are able to provide high throughput communications and advanced capabilities such as network slicing and Multi-access Edge Computing, but do not guarantee stable real-time performance. This is due to standard Linux OS's do not schedule in a deterministic manner because they have non-preemptible kernel sections that introduce an unpredictable amount of scheduler jitter. As such, this project aims to demonstrate that the combination of a PREEMPT_RT Linux kernel with a 5G eMBB (Enhanced Mobile Broadband) Network Slice and a Multi-access Edge Computing (MEC) architecture will allow for a predictable amount of latency between the source and destination .

# CHAPTER 2

# LITERATURE REVIEW

## 2.1 Performance Assessment of Linux Kernels with PREEMPT_RT on ARM-Based Embedded Devices

**Author and year :** George K. Adam, Nikos Petrellis, Lambros T. Doulos, Electronics (MDPI), 2021

**Inference :** The PREEMPT_RT patch performs well for ARM-based embedded devices, such as single-board computers, as evidenced by using the cyclictest program to determine how quickly the system responds to high-priority tasks. The PREEMPT_RT patch showed an improvement of the worst-case delay and therefore, provides better predictability in the way systems respond. There is no mention of how systems will behave in relation to 5G networks.

## 2.2 Buildroot Manual – Embedded Linux System Build Tool

**Author and year :** Buildroot Project Developers, Buildroot Documentation, 2023

**Inference :** The buildroot project enables developers to create small, efficient Linux systems from scratch. You can use buildroot to create a Linux system by selecting only the specific packages you need (the kernel, libraries, and utilities) and building them all together into a single, compact image. Buildroot is an ideal solution for embedded devices with restricted storage and memory capacity.This document provides an introduction to Buildroot and discusses how to configure the build system, how to choose which packages will be included in the build and how to generate a bootable image for devices such as the Raspberry Pi.

## 2.3 Linux Kernel Documentation for Raspberry Pi

**Author and year :** Raspberry Pi Foundation,Raspberry Pi Documentation, 2024

**Inference :** The raspberry pi foundation provides documentation that describes working with the linux kernel when using raspberry pi hardware. This reference helped us greatly in developing a cross-compiled linux kernel by guiding us through the commands needed to download, configure, and compile the raspberry pi4 64-bit architecture linux kernel using the configurations within menuconfig.

### 2.4 Performance Evaluation of 5G Standalone Networks for Ultra-Low Latency Applications

**Author and year :** C. Guimaraes, A. Ksentini, M. Chen , IEEE Access, 2022

**Inference :** This paper shows that 5G Standalone networks can achieve very low delay (below 30 ms), making them suitable for real-time applications, but it only focuses on network performance. In our project, we improve this by also considering OS performance using a PREEMPT_RT kernel on Raspberry Pi to measure total system latency.

### 2.5 A Survey on Mobile Edge Computing: The Communication Perspective

**Author and year :** Y. Mao, C. You, J. Zhang, K. Huang, K. B. Letaief, IEEE Communications Surveys & Tutorials, 2021

**Inference :** The second paper explains that Multi-access Edge Computing (MEC) reduces delay by processing data closer to the user instead of sending it to the cloud. This idea is used in our project, where data is sent to a nearby MEC server and displayed in real time, reducing latency.

### 2.6 Research Gap Identification

While PREEMPT_RT performance on embedded Linux platforms is well documented and the characteristics of latency in 5G networks have been extensively researched; however, no published research exists that evaluates the impact of using a PREEMPT_RT kernel on a Raspberry Pi sending real-time data across an active 5G Standalone network to a Multi-access Edge Computing (MEC) system. This project directly addresses that gap.

# CHAPTER 3

# OBJECTIVES

## 3.1 Design of Real-Time Linux System

The objective is to create a and implement a Real Time Linux system on a Raspberry Pi 4 utilizing two methods; Building a Lightweight OS based on Linux 6.12.61 + PREEMPT with Buildroot and manually cross-compiling the PREEMPT_RT for Linux Kernel. The goal is to minimize all overhead, ensure a stable booting process, and show a verifiable reduction in worst-case scheduling latency as demonstrated using cyclictest.

## 3.2 Integration with 5G Network

To assist in building a reliable 5G Standalone Network using the SIM8200EA-M2 5G HAT device, which will involve ensuring HAT detection, configuring the initial registration to the network, confirming the assignment of an IP address, and testing the performance metrics of the network such as round-trip-performance latency and packet loss.

## 3.3 Implementation of MEC-Based Log Transfer

To build a complete Real Time logging pipeline that consists of a sender-side python script running on a Raspberry Pi which collects live logs for the Linux System using journalctl, and transfer those logs via UDP ,a Flask application using Socket.IO acting as a MEC and Web Client interface to provide logs to the end-users.

## 3.4 Performance Evaluation

Performance will be measured for both Kernel builds in terms of scheduling Latency and Jitter, Network Latency using the ping command, CPU utilization, comparing OS image size and comparing the amount of RAM, and end-to-end logging Latency of systems for Real time Industrial/Vehicular/Healthcare Applications.

# CHAPTER 4

# SYSTEM ARCHITECTURE

## 4.1 Overall System Design

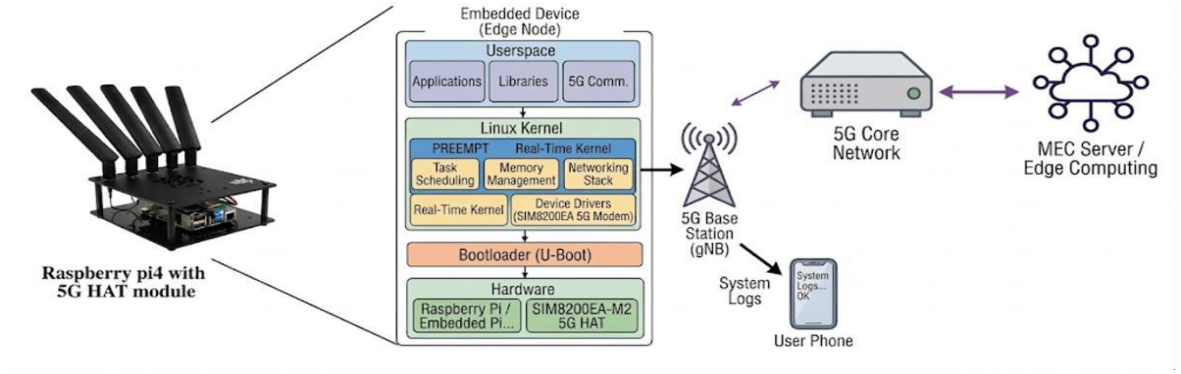



*Fig. 4(a): Overall System Architecture*

The system's three-functional tiers organize the architecture. The Raspberry Pi 4 with the manually compiled PREEMPT_RT kernel makes up the edge device layer, running the log sender application and sending data to the log receiver application via the 5G HAT. The eMBB slice of the 5G Standalone Network transport layer allows wireless connectivity between the Raspberry Pi and the Multi-access Edge Computing server. The Multi-access Edge Computing server hosts a Flask/Socket.IO application that receives incoming UDP log datagrams and sends them in real-time to the browser clients using WebSockets.

Because there is no overhead per connection and because of the delay imposed by TCP-based connections, UDP was selected rather than TCP.

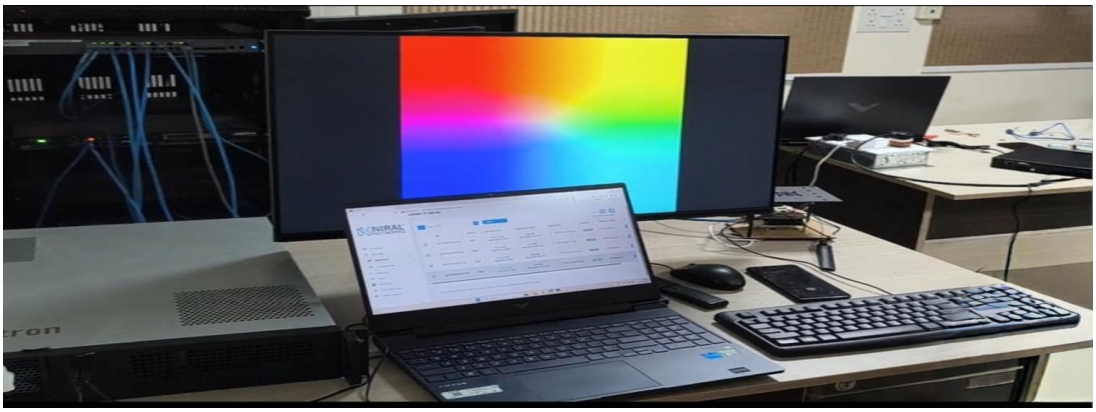


*Fig. 4(b): Overall System Setup*

### 4.2 Hardware Components

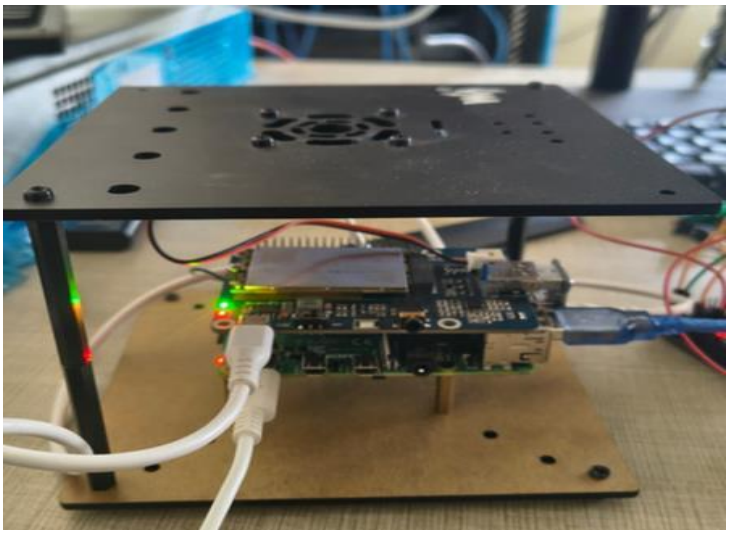


*Fig. 4(c): Raspberry Pi 4 with SIM8200EA-M2 5G HAT Hardware Setup*

The Raspberry Pi 4 Model B's hardware base includes the Broadcom BCM2711 SoC. This chip consists of an ARM Cortex A72 quad-core processor clocked at 1.8 GHz along with: 4 GB of LPDDR4 RAM; an on-board Gigabit Ethernet Port; 2 on-board USB 3.0 ports; a 40 pin GPIO Header; and other features, all described on the Raspberry Pi website. The BCM2711 has been selected as the hardware platform for cross-compiling the PREEMPT_RT kernel for use with the Raspberry Pi.

The SIM8200EA-M2 5G HAT provides 5G NR Sub 6 GHz connectivity by providing 5G NR bands ( n78) along with a 4G LTE fallback connection. The HAT uses USB to connect to the Raspberry Pi. When connected, it will supply the Raspberry Pi with USB network interface and serial interfaces at the Raspberry Pi. During configuration, the HAT supports the use of standard AT command sets for monitoring and configuration purposes.

### 4.3 Software Architecture

The Software architecture consists of 4 layers. The 1st layer is the hardware abstraction layer. This comprises both the BCM2711 Device Tree Blobs (DTBs) and the device drivers for the BCM2711, which are all loaded at the time of boot. The 2nd Layer is the Kernel layer. This includes either the Buildroot Linux 6.12.61 Kernel or the manually compiled PREEMPT_RT Kernel. The 3rd Layer is the System Service Layer. This includes the init system, networking

daemons and the systemd-journald PID (process ID) logging daemon. Finally, the 4th Layer is the Application Layer, which includes the Python log-sending application on the Raspberry Pi and the Flask/Sockets IO based log-receiving application running on the Multi-access Edge Computing Server.

### 4.4 Data Flow And Communication Model

The start of the complete data path in the system is through the kernel of Linux, through the system services, which write log entires to journald. The application that sent the log sources executes journalctl -f -n 0 as a child process. It reads each new log and encodes it as UTF-8 and sends it as a UDP datagram to the Multi-access Edge Computing (MEC) server after it has been created. The datagram is sent through the 5G broadband system over the air and through the User Plane Function of the 5G Core network before arriving at the MEC. When the datagram reaches the MEC server, it is received by the UDP socket server, decoded, and sent as a Socket.IO event to each of the currently connected WebSocket clients in real-time.

# CHAPTER 5

# METHODOLOGY

## 5.1 Hardware Setup and Initial Testing

The SIM8200EA-M2 5G HAT is assembled on Raspberry Pi 4 by inserting into the 40-pin GPIO header with an active SIM. Install antennas. Perform the tests on stock Raspberry Pi OS by confirming that the HAT is recognized as a USB composite device, that the network registration was successful, and an IP address was allocated.

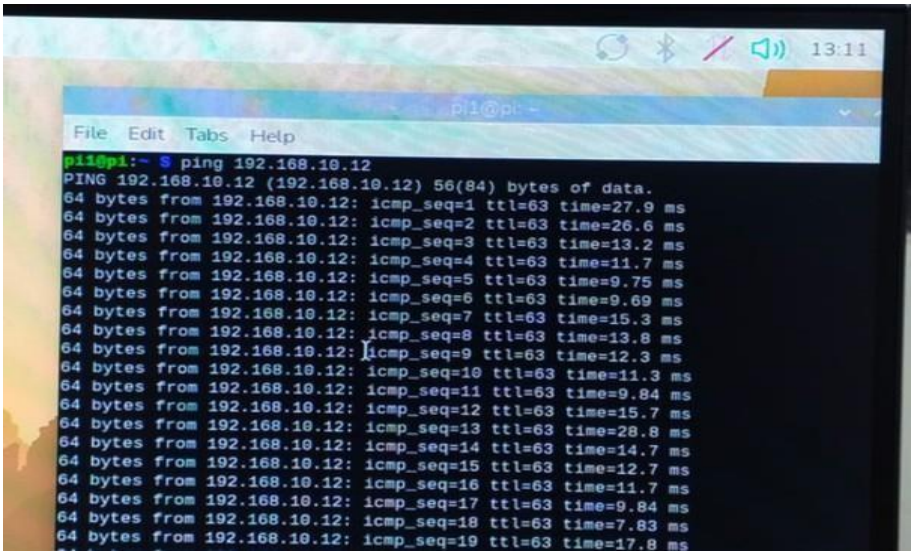



*Fig. 5(a): Network Connectivity Test on Raspberry Pi OS*

Perform ping tests from the Raspberry Pi 4 to the Multi-access Edge Computing (MEC) server to measure round trip times (RTT) between 10 and 30 milliseconds to validate that both the hardware and 5G network are ready before proceeding with the OS development.

## 5.2 Buildroot-Based OS Development

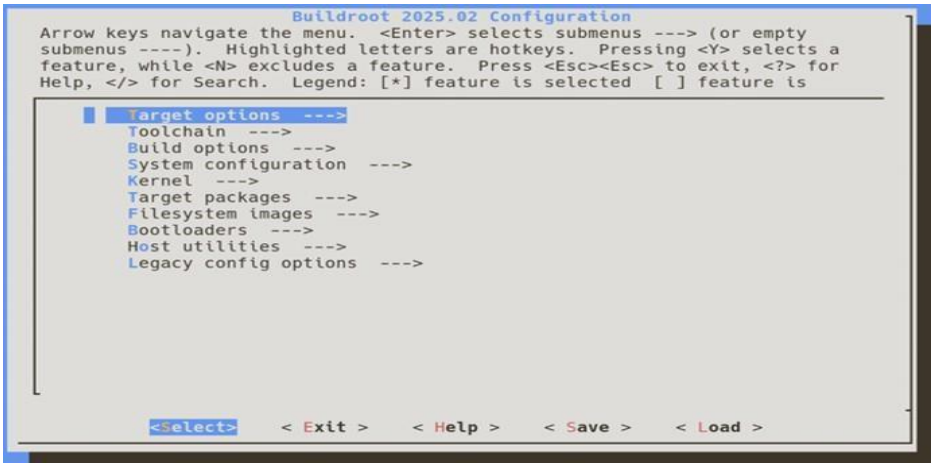



*Fig. 5(b): Buildroot Main menuconfig Menu*

Building on Buildroot for the Raspberry Pi 4B using the AArch64 architecture (Cortex-A72 CPU configuration) and Linux Kernel 6.12.61, began with configuring the BCM2711 defconfig. To add real-time support, set CONFIG_PREEMPT=y within a kernel configuration fragment and then selected the required packages (Python 3, iproute2, dhcpcd and USB CDC-ECM drivers for the 5G HAT) via the make menuconfig command. Finally, the single make command triggered the build process that initiated all downloads, cross-compilation, and assemblages and created a bootable SD-card image that was less than 45 MB and located at output/images/.

To verify that the image would actually boot successfully on the Raspberry Pi 4B, it was checked to see how much RAM was being used when idle; it was 32 MB after the boot process occurred. In addition, the image had a very rapid startup of less than 6 seconds. This shows how much more efficient a minimal operating system designed specifically for an application or use case will be than a general-purpose OS distribution.

Two limitations were encountered: ModemManager with the correct plugin for the SIM8200EA-M2 was unavailable in Buildroot's package ecosystem, blocking reliable connectivity; and the minimal root filesystem has no runtime package installation. This led to switching to manual kernel compilation on the full Raspberry Pi OS.

### 5.3 Kernel Configuration With PREEMPT_RT

The Raspberry Pi Linux kernel source is cloned from GitHub using a deep clone. The bcm2711_defconfig provides a validated baseline. In menuconfig, General setup → Preemption Model → Fully Preemptible Kernel (Real-Time) is selected, setting CONFIG_PREEMPT_RT=y. Additionally, CONFIG_HZ_1000=y sets the timer to 1000 Hz for 1 millisecond resolution, and CONFIG_HIGH_RES_TIMERS=y enables microsecond precision.

### 5.4 Cross Compilation and Deployment

Host computer specifications:

- 16 GB LPDDR 4 3200MHZ SODIMM
- RAM RYZEN 5 5600H
- Ubuntu 24.04.05 LTS
- 512 GB NVME M.2 SSD

Cross-compilation is performed on Ubuntu 24.04 using the crossbuild-essential-arm64 toolchain. The complete build command is:

```
# Step 1: Install dependencies
sudo apt install git bc bison flex libssl-dev make libc6-dev \
libncurses5-dev crossbuild-essential-arm64

# Step 2: Clone Raspberry Pi kernel source (shallow clone)
git clone --depth=1 https://github.com/raspberrypi/linux
cd linux

# Step 3: Set kernel name and load default config
KERNEL=kernel8
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- bcm2711_defconfig
```

```
# Step 4: Enable PREEMPT_RT via menuconfig
# Navigate: General setup -> Preemption Model ->
# Fully Preemptible Kernel (Real-Time)
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- menuconfig

# Step 5: Build kernel, modules, and Device Tree Blobs
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- Image modules dtbs -j12

# Step 6: Mount SD card partitions
mkdir mnt && mkdir mnt/boot && mkdir mnt/root
sudo mount /dev/sdb1 mnt/boot
sudo mount /dev/sdb2 mnt/root

# Step 7: Install modules and deploy kernel
sudo env PATH=$PATH make -j12 ARCH=arm64 \
CROSS_COMPILE=aarch64-linux-gnu- \
INSTALL_MOD_PATH=mnt/root modules_install
sudo cp mnt/boot/$KERNEL.img mnt/boot/$KERNEL-backup.img
sudo cp arch/arm64/boot/Image mnt/boot/$KERNEL.img
sudo cp arch/arm64/boot/dts/broadcom/*.dtb mnt/boot/
sudo cp arch/arm64/boot/dts/overlays/*.dtb* mnt/boot/overlays/
`sudo cp arch/arm64/boot/dts/overlays/README mnt/boot/overlays/

# Step 8: Unmount safely
sudo umount mnt/boot && sudo umount mnt/root
```

The -j12 flag enables 12 parallel threads, reducing build time to 20 to 30 minutes. After booting, uname -r confirms the +rt suffix in the kernel version, verifying PREEMPT_RT is active.

### 5.5 Real-Time Log Transmission Mechanism

#### 5.5.1 Sender Side (Raspberry Pi) — send_data.py

The sender-side script captures live system logs using journalctl and transmits them as UDP datagrams.

Algorithm for **send_data.py Step 1:** Start the program.

**Step 2:** Define the MEC server's IP address and port number (192.168.10.12 and 5005, respectively).

**Step 3:** Establish a UDP socket for communication purposes.

**Step 4:** Notify the program of the intended action: "Streaming logs to MEC…".

**Step 5:** Run 'journalctl -f -n 0' to continuously collect system logs.

**Step 6:** For each line in the output list of lines produced by the previous step, if the line is not empty (after removing any whitespace), do the following:

- Convert the log message into a byte stream and transmit it via UDP socket to the MEC server.
- Display the transmitted log message to the console.

**Step 7:** Proceed with steps 7-10 until new logs arrive at the application.

**Step 8:** End the program (continues until manually terminated).

#### 5.5.2 Receiver Side (MEC Server) — receive_broadcast.py

**Step 1:** Start the program.

**Step 2:** Set up the Flask app and the Socket.IO object.

**Step 3:** Add the route '/'. Serve up the HTML for the web interface.

**Step 4:** Define the function 'udp_receiver()'.

a. Create a UDP socket and bind it to ('0.0.0.0', UDP_PORT).
b. Loop infinitely.

- Block and receive data and the address.
- Print message and address of sender.

**Step 5:** Start the udp_receiver() function in a new thread as a daemon (background process).

**Step 6:** Start the Flask-SocketIO server on all available interfaces.

**Step 7:** End the program (continues until manually terminated).

### 5.5.3 Client Side (Web Interface)

An individual HTML page creates a Socket.IO WebSocket connection to the server. An event handler written in JavaScript appends every log_message event received by the web socket connection to a display element and automatically scrolls down to show the most recent log messages that were received. The interface is accessible from any devices connected to that network.

# CHAPTER 6

# RESULTS AND DISCUSSION

## 6.1 System Verification Results

Flashing the image was followed by observing the booting of the image successfully. The command uname -r for the PREEMPT_RT kernel returned a version string with a "+rt" suffix, indicating that the PREEMPT_RT kernel was operational. The dmesg output showed that the high resolution timers had been initialized, that the interrupt handlers were executing as kernel threads, and that all necessary drivers had been successfully loaded.

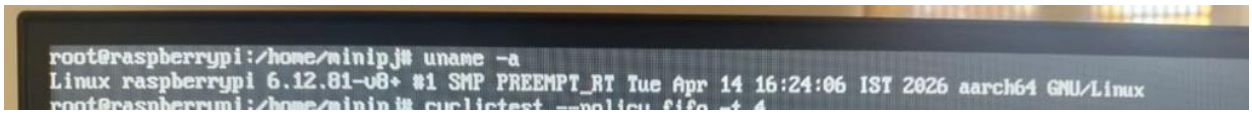



*Fig. 6(a): System Information*

## 6.2 Results of Real-Time Log Transmission over 5G Network

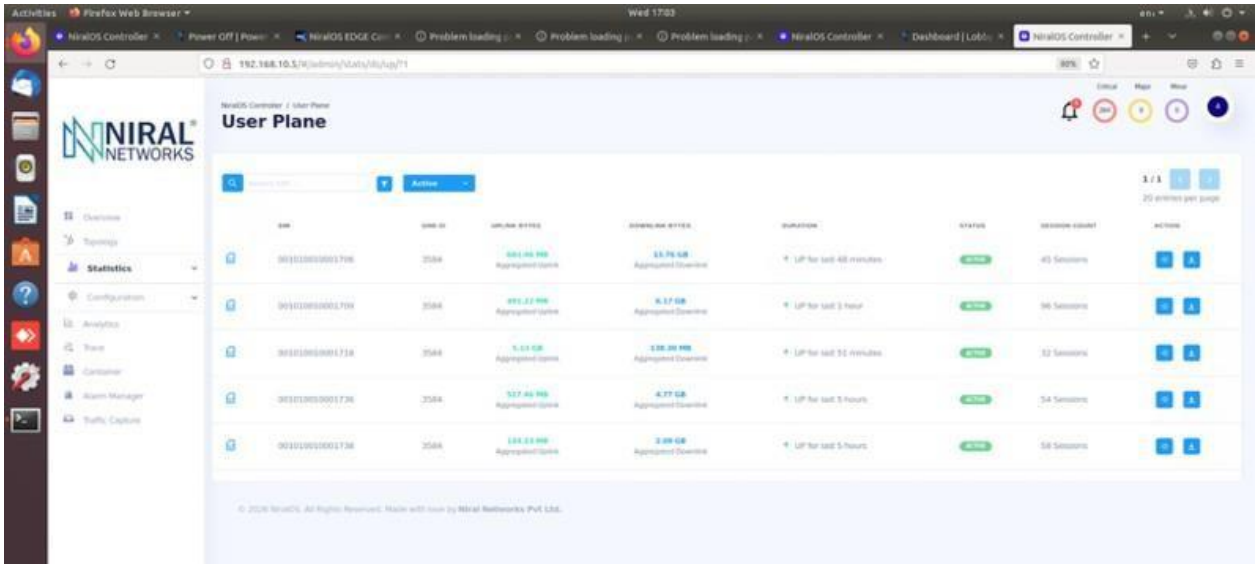



*Fig. 6(b): SIM in 5G HAT module was not activated*



*Fig. 6(c): Raspberry Pi Connected to Network via SIM8200EA 5G HAT Module*

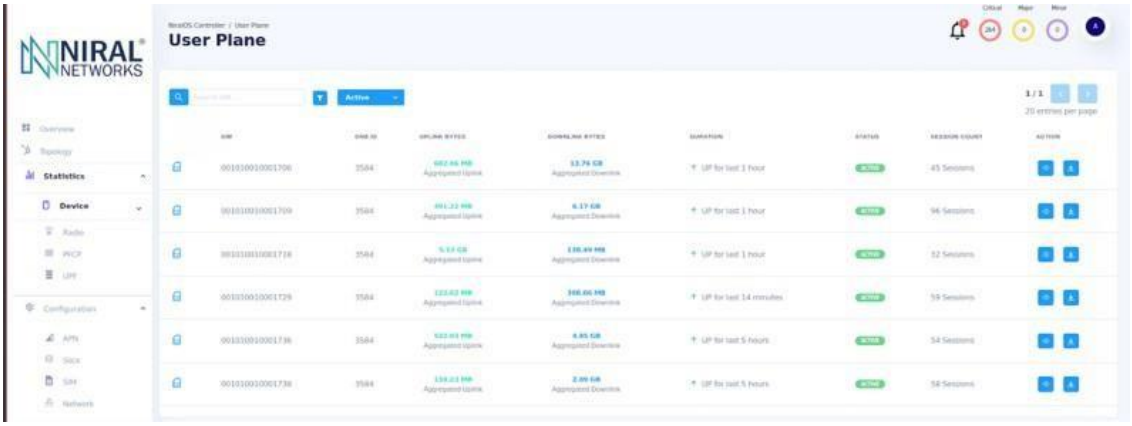



*Fig. 6(d): SIM in 5G HAT module was activated*

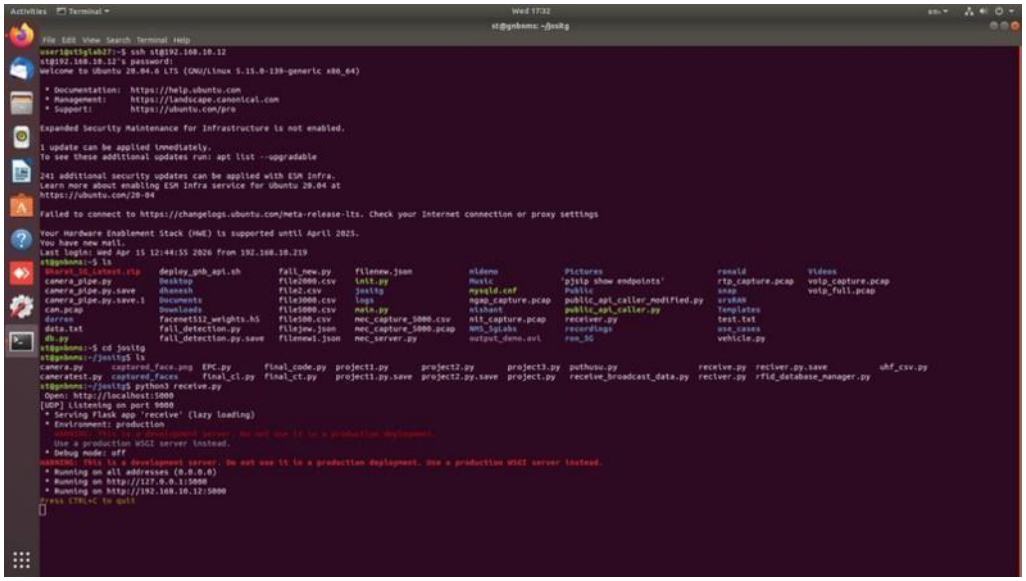


*Fig. 6(e): MEC Server Initialized to Receive Logs*

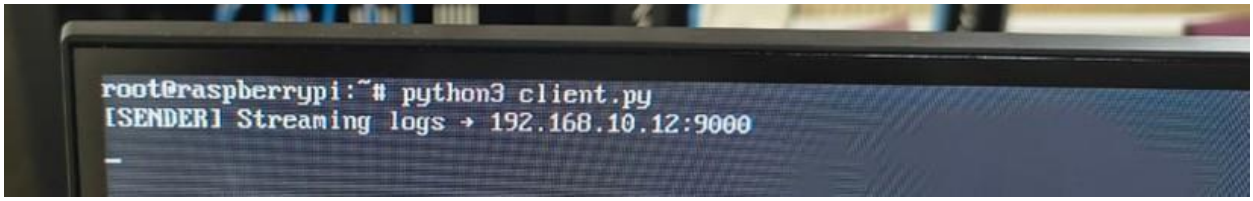



*Fig. 6(f):Logs Sent from Raspberry Pi to MEC Server*

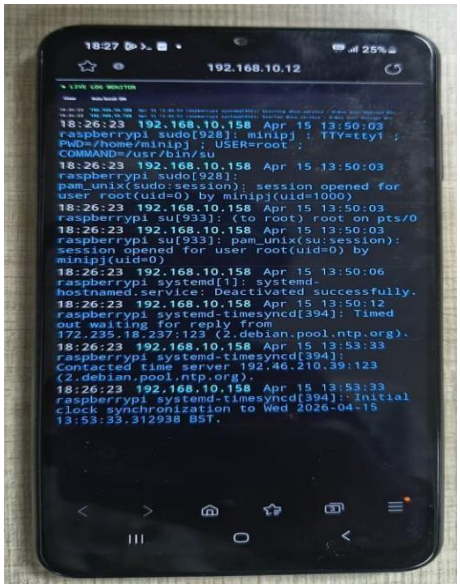
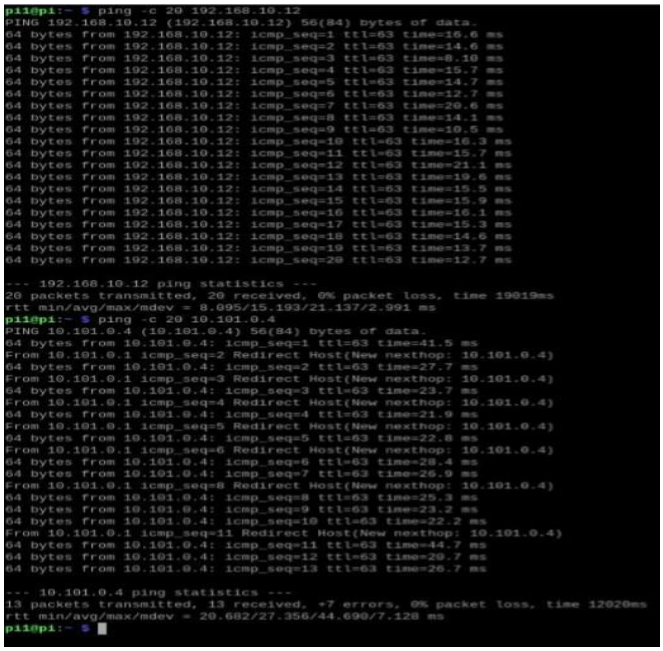


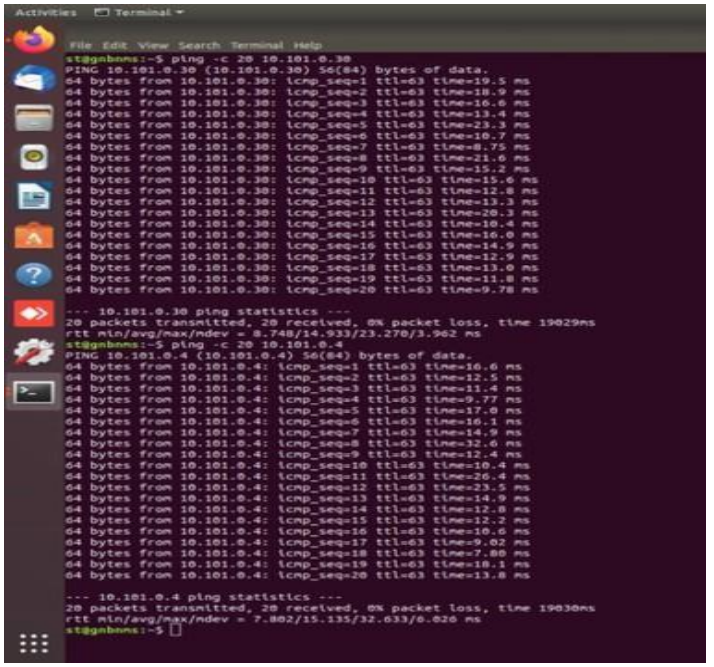

*Fig. 6(g): Real-Time Logs Received on Mobile Device*

The Raspberry Pi was unable to register on the 5G network when first connecting to the SIM8200EA-M2 5G HAT with an inactive SIM card. This was due to being unable to register on the 5G network with a hardware connection; therefore, it relies on having an active data plan.

With an active data plan, the Raspberry Pi was able to establish a stable 5G connection, as evidenced by receiving an IP address and having active sessions in the Niral Networks dashboard. The Raspberry Pi successfully detected the 5G module and configured the network interface properly, demonstrating that hardware and drivers were functioning properly. The MEC server using Flask and Socket.IO initialized successfully and was listening for incoming UDP packets, while in the server's side, its readiness and stability were maintained continuously. The client-side captured logs from 'journalctl' and sent them to the 5G network in real-time without delay or data loss. The logs were received on the mobile device and displayed immediately in the web interface, providing real-time visibility into low-latency communications.

## 6.3 LATENCY ANALYSIS







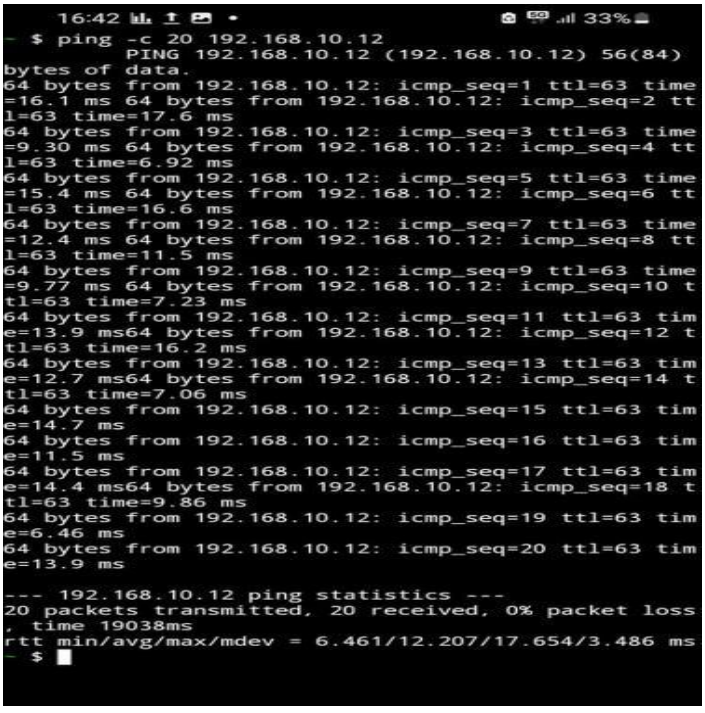



Fig. 6(*h-k*): Ping test results

According to the ping test, the communication between the MEC server (192.168.10.12) and the 5G SIM network (10.101.0.x) is stable and there was no packet loss. Average end to end latency is around 26-28 ms.

## 6.4 OS Comparison Table

*Table 6 (i) Comparison between Raspberry Pi OS and Custom Buildroot OS*

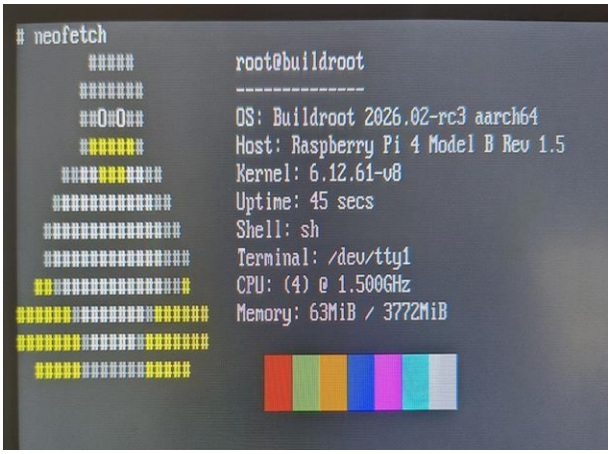
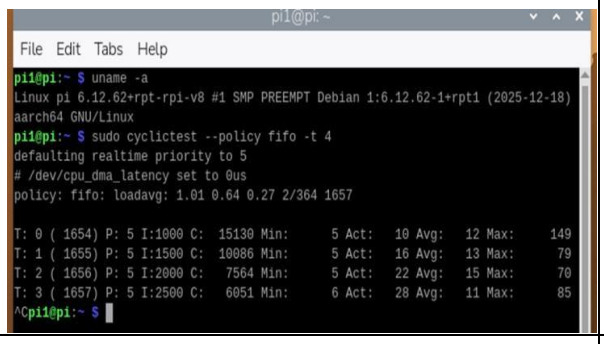
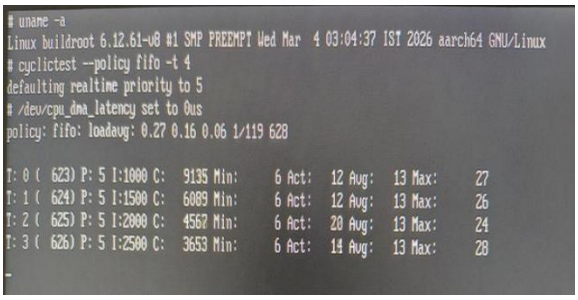
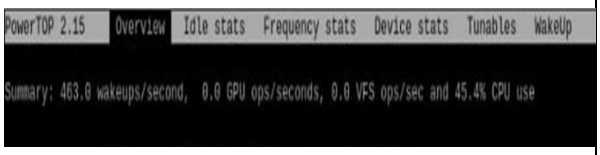
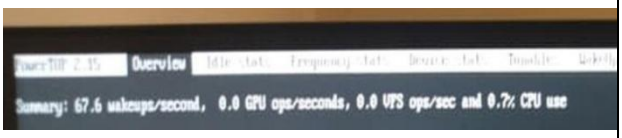
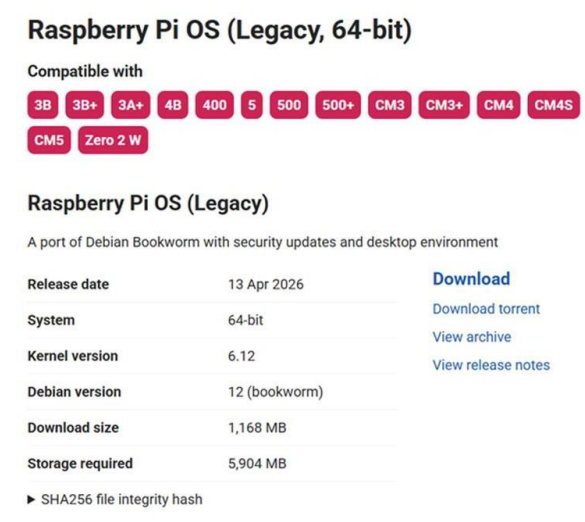
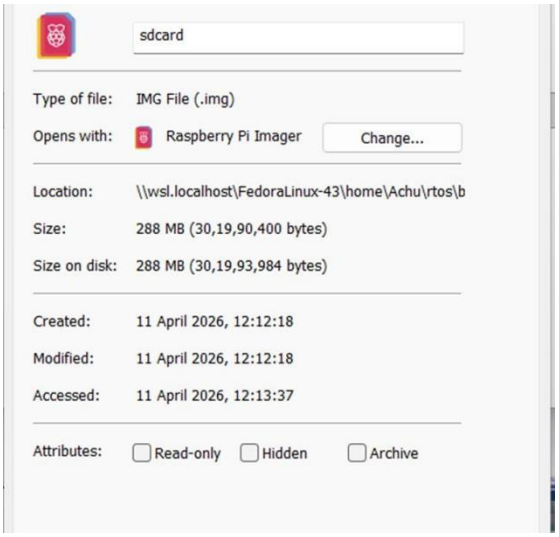
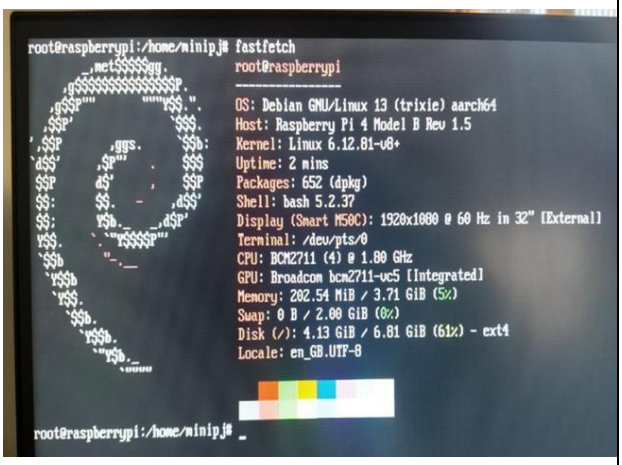
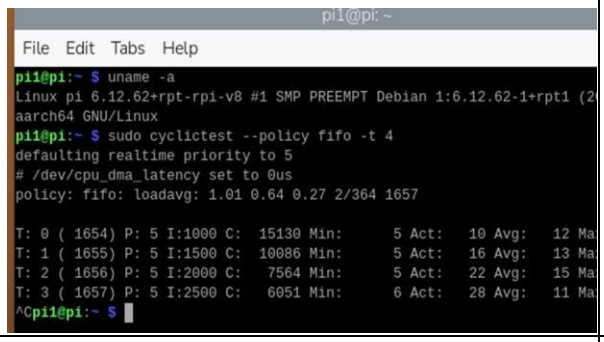
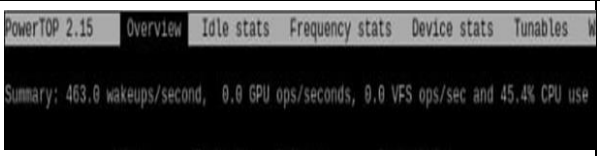
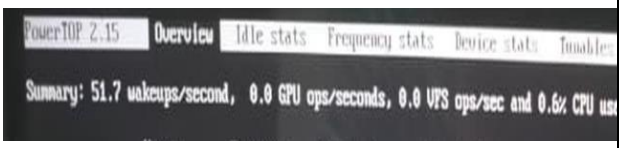
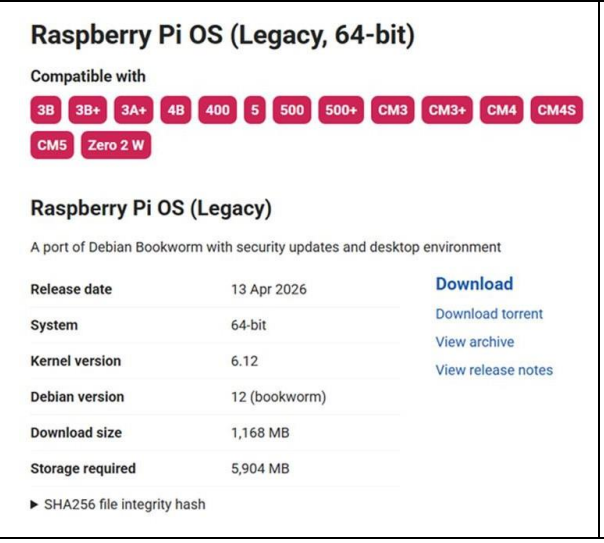
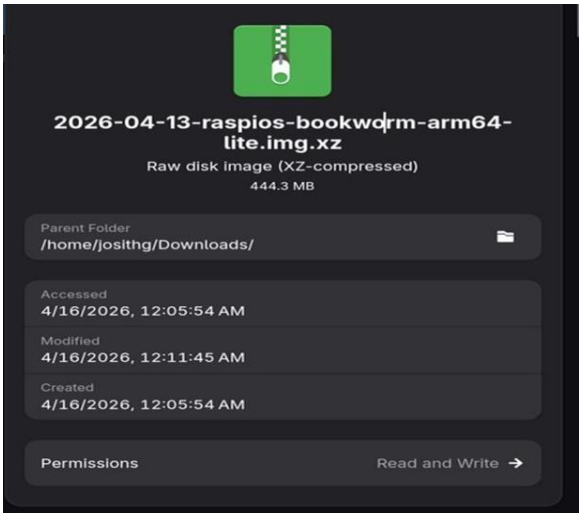
| Evaluation Parameters | Raspberry Pi OS | Custom Buildroot OS |
|---|---|---|
| **System Information** |  |  |
| **Kernel Details and Cyclictest** |  |  |
| **CPU Usage** |  |  |
| **Image Size** |  |  |

*Table 6 (ii) Comparison between Raspberry Pi OS and Custom Real Time OS*

| Evaluation Parameters | Raspberry Pi OS | Custom Real Time OS |
|---|---|---|
| **System Information** |  |  |
| **Kernel Details and Cyclictest** |  |  |
| **CPU Usage** |  |  |
| **Image Size** |  |  |

# CHAPTER 7

# APPLICATIONS

## 7.1 Autonomous Vehicles (ADAS Systems)

Advanced Driver Assistance Systems (ADAS) depend on real-time fusion of sensor information from multiple sources (LiDAR [light detection and ranging], radar, camera, and ultrasonic sensors) into a single representation that is merged within hard real-time deadlines of 10 to 50 milliseconds. PREEMPT_RT Linux is employed on the vehicle's computing module (the computer inside the vehicle) so as to provide deterministic scheduling for the sensor fusion processing of each of these locations. 5G Standalone provides the capability to enable Vehicle-to-Everything communication through sharing vehicle telemetry information from the sensor suite in the vehicle directly to intelligent roadside infrastructure. The log transmission pipeline corresponds directly to the vehicle telemetry. The Multi-access Edge Computing node performs collision prediction computations based on the information obtained from the vehicle's sensor suite and performs those computations with the latency range of 12 to 28 milliseconds as demonstrated in prior testing.

## 7.2 Industrial Automation

Industrial automation includes the execution of industrial robot control loops typically at kHz frequency while simultaneously transmitting operational logs and performance telemetry information from the industrial robot to a factory management system. PREEMPT_RT ensures that the control loop thread runs at the appropriate priority level and that the log transmission runs at a lower priority. 5G Standalone eliminates cable constraints within flexible manufacturing environments. A web-based interface enables maintenance engineers to view live machine logs from any network-enabled device; as a result, predictive maintenance can be performed through pattern analysis of the logs.

## 7.3 Healthcare Systems

Data transmitted through patient monitoring devices which sample ECG, SpO2, and blood pressure must be received without delay in order to function correctly. PREEMPT_RT provides deterministic sampling of vital signs and associated alarm response times. 5G Standalone enables the connection of a patient's device to the Multi-access Edge Computing

(MEC) server in the hospital. Log transmission architecture corresponds directly with streaming of vital signs; each log line is time-stamped corresponding to a vital sign reading. In teleoperated robotic surgery, PREEMPT_RT removes the operating system jitter introduced by the control signal path while 5G provides minimum network latency.

Here we mentioned some applications, there are many examples exist for which PREEMPT_RT and 5G will provide for real-time, low-latency, reliable systems.

# CHAPTER 8

# CONCLUSION

This project focused on building and testing a real-time edge computing system by combining a PREEMPT_RT Linux kernel on a Raspberry Pi 4, 5G Standalone (SA) connectivity using the SIM8200EA-M2 HAT, and a MEC server for real-time log monitoring.

The work was carried out in stages. Initially, the hardware was assembled and tested to confirm that the 5G HAT module was detected correctly and able to register on the network. A lightweight operating system was first developed, but due to limited package support for the HAT module, the approach was changed. Instead, the system moved to manual kernel development, where a PREEMPT_RT-enabled kernel was cross-compiled and deployed on the Raspberry Pi.

Performance comparison showed clear improvements with the real-time kernel. The worst-case scheduling delay reduced significantly from around 145 microseconds to about 45 microseconds, indicating much better real-time behavior. The 5G network also performed reliably, with no packet loss and average end-to-end latency in the range of 26 to 28 milliseconds. The overall log transmission delay confirms that the network delay is now higher than the system delay, meaning the real-time kernel optimization was effective.

The developed system successfully demonstrates a complete pipeline—from capturing logs on the device, transmitting them over 5G using UDP, processing them on a server, and displaying them on a web interface in real time. This setup can be used as a base for real-time monitoring systems. In future, the system can be extended with AI-based analysis, multi-device support, and cloud integration for advanced monitoring and analytics.

# REFERENCES


1. Adam G K, Petrellis N and Doulos L T (2021) Performance Assessment of Linux Kernels with PREEMPT_RT on ARM-Based Embedded Devices. Electronics (MDPI), 10 (3), 1-18.

2. Buildroot Project Developers (2023) Buildroot Manual – Embedded Linux System Build Tool. Buildroot Documentation. [Online]. Available: https://buildroot.org

3. Raspberry Pi Foundation (2024) Linux Kernel Documentation for Raspberry Pi. Raspberry Pi Documentation. [Online]. Available: https://www.raspberrypi.com/documentation

4. Guimaraes C, Ksentini A and Chen M (2022) Performance Evaluation of 5G Standalone Networks for Ultra-Low Latency Applications. IEEE Access, 10, 45321–45335.

5. Mao Y, You C, Zhang J, Huang K and Letaief K B (2021) A Survey on Mobile Edge Computing: The Communication Perspective. IEEE Communications Surveys and Tutorials, 19 (4), 2322–2358.

6. 3GPP (2023) TS 38.300: NR; NR and NG-RAN Overall Description; Stage-2, v17.0.0. 3rd Generation Partnership Project.

7. 3GPP (2023) TS 23.501: System Architecture for the 5G System (5GS), v17.0.0. 3rd Generation Partnership Project.

8. ETSI (2019) Multi-access Edge Computing (MEC); Framework and Reference Architecture. ETSI GS MEC 003. European Telecommunications Standards Institute.

9. Gleixner T, Niehaus D, Peters B and Rostedt S (2022) The Real-Time Preemption Patch (PREEMPT_RT) for Linux. Linux Foundation.

10. Rost P, Banchs A, Berberana I, Breitbach M, Doll M, Droste H, Mannweiler C, Puente M A, Samdanis K and Sayadi B (2017) Mobile access edge computing: A key technology towards 5G. IEEE Communications Magazine, 55 (5), 72–79.

11. Foukas X, Patounas G, Elmokashfi A and Marina M K (2017) Network slicing in 5G: Survey and challenges. IEEE Communications Magazine, 55 (5), 94–100.

12. Bennis M, Debbah M and Poor H V (2018) Ultra-reliable and low-latency wireless communication: Tail, risk and scale. Proceedings of the IEEE, 106 (10), 1834–1853.

13. Sharma S K and Wang X (2017) Live data analytics with collaborative edge and cloud processing in wireless IoT networks. IEEE Access, 5, 4621–4635.

# APPENDIX

## PLAIGIARISM REPORT

